МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

СОВРЕМЕННЫЕ МЕТОДЫ ВЁРСТКИ · FLEXBOX · CSS GRID

Проектирование и разработка интерфейсов пользователя

Преподаватель

Коляда Никита Владимирович

Тема 1.3

ПЛАН ЗАНЯТИЯ

1. Раскладка в ряд: магия Flexbox

Как легко выстроить элементы в одну линию и выровнять их.

2. Сетки из карточек: CSS Grid

Двумерная сетка: делим экран на равные колонки как плитку шоколада.

3. Адаптивность и переменные

Как подстроить сайт под экран смартфона и управлять цветами через :root.

4. Практика: каталог рецептов

Пошагово верстаем отзывчивую витрину блюд с кнопками и отступами.

5. Итоги занятия и разбор типичных ошибок

Разбор типичных ошибок верстки, методы отладки и закрепление материала

ЗАДАЧА РАСКЛАДКИ

css/style.css (поведение блоков)

```css
/* Обычные теги <div> встают друг под другом */
.card {
  display: block;  /* занимает 100% ширины */
  background: #ffffff;
  padding: 16px;
}

/* А нам нужно поставить их рядом в один ряд! */
/* Для этого в современном CSS есть Flexbox */
```

ПОЧЕМУ БЛОКИ ПАДАЮТ ВНИЗ

- **Блоки по умолчанию**
  Теги , <header>, <section> — блочные. Они жадно занимают всю ширину строки от края до края.
- **Стена из кирпичиков**
  Следующий элемент физически не может встать рядом и падает на новую строчку вниз.
- **Задача для сайтов**
  В интерфейсах нам постоянно нужно ставить элементы в ряд: логотип и меню, кнопки или карточки.
- **Современное решение**
  Не нужно мучиться со сложными костылями — для раскладки в ряд в CSS есть идеальный инструмент Flexbox!

Почему обычные блоки не встают рядом друг с другом автоматически?

css/style.css (включаем ряд)

```css
/* Родительский блок становится флекс-контейнером */
.header {
  display: flex;  /* волшебная строчка */
}

/* Дочерние теги мгновенно встают в один ряд: */
/* [ Логотип ]  [ Меню ссылок ]  [ Кнопка ] */
```

КАК РАБОТАЕТ DISPLAY: FLEX

- **Пишется на родителе**
  Мы объявляем display: flex не на кнопках, а на их общем контейнере (например, на теге <header>).
- **Полочка для элементов**
  Все вложенные дети мгновенно слушаются и встают в одну аккуратную линию слева направо.
- **Контейнер и элементы**
  Родительский тег называют флекс-контейнером, а его прямых детей — флекс-элементами.
- **Всего одна строчка**
  Одно правило display: flex решает проблему горизонтальной раскладки без лишнего сложного кода.

На каком теге нужно написать display: flex — на карточке или на общем родителе?

НАПРАВЛЕНИЕ РЯДА

css/style.css (строка или колонка)

```css
/* 1. По умолчанию: ряд слева направо */
.menu {
  display: flex;
  flex-direction: row;  /* горизонталь */
}

/* 2. Вертикальный столбик: сверху вниз */
.sidebar {
  display: flex;
  flex-direction: column;  /* вертикаль */
}
```

УПРАВЛЕНИЕ НАПРАВЛЕНИЕМ

- **Значение row (по умолчанию)**
  Элементы выстраиваются в горизонтальную цепочку слева направо (идеально для шапки сайта).

- **Значение column (колонка)**
  Поворачивает направление на 90°: элементы встают в аккуратный вертикальный столбик.

- **Где нужен column**
  Отлично подходит для боковых панелей сайта, карточки профиля (аватарка → имя → кнопка) и мобильного меню.

- **Легко переключать**
  На компьютере делаем row (в ряд), а на экране телефона меняем на column — интерфейс сразу удобен!

Как повернуть горизонтальный ряд ссылок в вертикальное мобильное меню?

css/style.css (justify-content)

```css
.site-header {
  display: flex;
  justify-content: space-between;
}

/* Другие популярные значения: */
/* flex-start  — собрать всех слева */
/* center      — собрать всех по центру */
/* flex-end    — прижать всех вправо */
```

РАСПРЕДЕЛЕНИЕ В СТРОКЕ

- **Суть свойства**
  justify-content управляет тем, как свободное место в строке распределяется между элементами.
- **Хит веба: space-between**
  Первый элемент прижимается к левому краю, последний — к правому, а между ними — равный отступ.
- **Идеально для шапки сайта**
  Логотип слева, кнопка входа справа — создается ровно одной красивой строчкой CSS.
- **Центрирование через center**
  Если нужно собрать кнопки или заголовок строго посредине экрана — пишем justify-content: center.

Какое значение justify-content расталкивает логотип и меню к противоположным краям?

ВЫРАВНИВАНИЕ ВЫСОТЫ

css/style.css (align-items)

```css
.header-inner {
  display: flex;
  align-items: center;  /* ровняем по центру */
}

/* Другие значения: */
/* stretch    — растянуть по высоте (по умолчанию) */
/* flex-start — выровнять по верхнему краю */
/* flex-end   — выровнять по нижнему краю */
```

ВЕРТИКАЛЬНЫЙ БАЛАНС

- **Проблема разной высоты**
  Логотип высокий (40px), а ссылки меню мелкие (16px). Без выравнивания текст некрасиво прилипнет к верху.

- **Спасительный center**
  Свойство align-items: center проводит невидимую среднюю линию и центрирует все элементы по высоте.

- **Поведение stretch**
  По умолчанию элементы растягиваются на всю высоту самого высокого соседа в строке.

- **Конец ручным отступам**
  Больше не нужно наугад подбирать margin-top в пикселях, чтобы выровнять иконку и текст!

Как вертикально выровнять по центру круглую аватарку и текст рядом с ней?

css/style.css (gap и перенос)

```css
.tags-list {
  display: flex;
  gap: 16px;          /* ровный зазор между тегами */
  flex-wrap: wrap;     /* перенос на новую строку */
}

/* Больше никаких лишних отступов у последнего элемента! */
```

РАССТОЯНИЕ МЕЖДУ ЭЛЕМЕНТАМИ

- **Свойство gap — суперсила**
  Задает отступ строго между соседними элементами, не создавая лишних полей по краям.
- **Без костылей**
  Раньше приходилось писать margin-right и обнулять его у последнего элемента. С gap всё работает сразу.
- **Перенос flex-wrap: wrap**
  Если карточек или кнопок слишком много для экрана, они аккуратно перетекают на следующую строчку.
- **Защита от сплющивания**
  Без wrap элементы начнут сжиматься в узкие полоски, пытаясь любой ценой уместиться в одну строку.

Чем свойство gap удобнее старого способа с margin-right у каждого элемента?

css/style.css (сетка каталога)

```css
/* Контейнер каталога рецептов */
.recipe-catalog {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;                /* зазор 20px между ячейками */
}

/* Браузер сам разделит экран на 3 равные колонки! */
```

КОГДА ОДНОЙ СТРОКИ МАЛО

- **Flexbox vs Grid**
  Flexbox идеален для одной линии (шапка, меню). Но если у нас витрина из 9 карточек — берем CSS Grid.
- **Плитка шоколада (2D)**
  Grid умеет строить таблицы: одновременно управлять и строками, и столбцами макета.
- **Что такое 1fr**
  Единица fr (fraction) означает «одна равная доля». repeat(3, 1fr) мгновенно делит ширину на 3 равные колонки.
- **Без калькулятора**
  Не нужно вычислять проценты 33.333% и вычитать отступы — браузер производит все расчеты сам.

Что произойдет на экране, если написать grid-template-columns: repeat(3, 1fr)?

css/style.css (акцентная карточка)

```css
/* Обычная карточка занимает 1 колонку */
.recipe-card { background: #ffffff; }

/* Главный рецепт дня растягиваем на 2 колонки! */
.recipe-card.featured {
  grid-column: span 2;  /* занять 2 ячейки */
}
```

АКЦЕНТЫ В СЕТКЕ

- **Свойство grid-column: span 2**
  Приказывает элементу растянуться на ширину сразу двух соседних колонок сетки.
- **Где это нужно**
  Большой рекламный баннер акции, «рецепт дня» или крупная главная новость в каталоге.
- **Сетка подстраивается**
  Соседние карточки автоматически сдвигаются на свободные места, макет не ломается.
- **Свобода раскладки**
  В Grid легко создавать интересные журнальные композиции без сложных вложенных блоков.

Как заставить карточку занять ширину двух колонок в сетке каталога?

СХЕМА: ВЫБОР ИНСТРУМЕНТА

1. FLEXBOX (1D)

Полочка: элементы в один ряд



В одну линию (1D):

Идеально для шапки сайта, меню со ссылками или ряда кнопок.

- Направление: элементы идут слева направо по главной оси.
- **Свойство в CSS:**

```
display: flex; gap: 8px;
```

2. CSS GRID (2D)

Плитка шоколада: строки и колонки



Сетка карточек (2D):

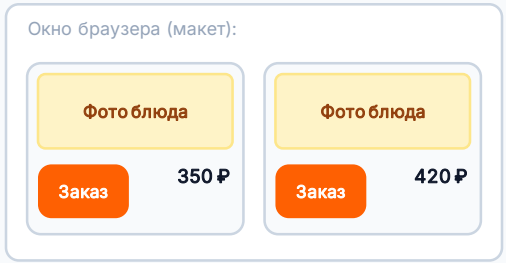
Управляет сразу строками и колонками для каталога и галереи.

- Двумерность: сетка держит строки (Y) и колонки (X) одновременно.
- **Свойство в CSS:**

```
grid-template-columns: repeat(3, 1fr);
```

3. ДРУЖБА В UI

Работают в одной команде!



Командная работа:

Снаружи Grid держит колонки, а Flexbox внутри ровняет кнопку и цену.

- Связка: Grid строит общую сетку, Flexbox ровняет кнопку в карточке.
- **Свойство в CSS:**

```
Grid снаружи + Flexbox внутри карточки
```

В каком месте верстки лучше взять Grid, а где обязательно поставить Flexbox?

css/style.css (медиа-запрос)

```css
/* На обычном мониторе — 3 колонки */
.catalog {
  grid-template-columns: repeat(3, 1fr);
}

/* ЕСЛИ экран уже 768px (смартфон или планшет): */
@media (max-width: 768px) {
  .catalog {
    grid-template-columns: 1fr; /* всего 1 колонка */
  }
}
```

АДАПТИВНОСТЬ ПОД ЭКРАН

- **Зачем это нужно**
  На мониторе 3 колонки рецептов смотрятся отлично, а на узком телефоне они сожмутся в нечитаемую кашу.

- **Простое условие @media**
  Конструкция @media (max-width: 768px) означает: «Если экран уже 768px, примени эти правила».

- **Перестройка в 1fr**
  Мы просто меняем 3 колонки на 1fr — и карточки встают в 1 аккуратную колонку на весь экран телефона.

- **Минимум кода**
  Цвета, рамки и картинки переписывать не надо — внутри @media меняется только количество колонок.

Что означает условие @media (max-width: 768px) простыми словами?

css/style.css (дизайн-токены)

```css
/* Хранилище общих настроек всего сайта */
:root {
  --accent-color: #fe6002; /* фирменный оранжевый */
  --card-radius: 12px;    /* скругление углов */
}

/* Используем сохраненные значения через var() */
.btn {
  background-color: var(--accent-color);
  border-radius: var(--card-radius);
}
```

ЕДИНАЯ ПАЛИТРА ПРОЕКТА

- **Проблема повторов**
  Главный цвет проекта используется в кнопках, ссылках, иконках и рамках десятков разных элементов.

- **Что такое :root**
  Это специальное место в начале CSS-файла, где мы объявляем свои переменные со знаком -- в начале.

- **Команда var(--имя)**
  Подставляет сохраненное значение цвета или отступа в любое нужное CSS-свойство.

- **Магия за 1 секунду**
  Заказчик попросил сменить цвет сайта — меняем 1 строчку в :root, и весь сайт перекрашивается!

Почему хранить главный цвет в переменной :root удобнее, чем писать #fe6002 в 50 местах?

css/style.css (приоритеты)

```
/* 1. Селектор тега — самый слабый */
button { color: #888888; }

/* 2. Селектор класса — ВСЕГДА СИЛЬНЕЕ тега! */
.btn-order { color: #ffffff; } /* ПОБЕДИТ! */

/* 3. Если сила равна — побеждает то, что НИЖЕ */
.btn-order { background: #fe6002; }
```

КАК БРАУЗЕР РЕШАЕТ СПОРЫ

- **Когда возникает спор**
  Если к одной кнопке написано два разных цвета, браузеру нужно выбрать один победный.
- **Класс сильнее тега**
  Селектор по классу .btn всегда побеждает общий селектор тега button. Классы — наш главный инструмент.
- **Кто ниже — тот и главный**
  Если правила имеют одинаковую силу, браузер применит то, которое написано строчкой ниже в файле.
- **Наследование от предков**
  Шрифт и цвет текста автоматически передаются от родителя детям, их не нужно дублировать.

Какой цвет применится к кнопке с class='btn', если в CSS есть button { color: red; } и .btn { color: green; }?

css/style.css (опасно vs надежно)

```css
/* ❌ ОПАСНО: жесткий пиксельный размер */
.bad-card {
  width: 500px;   /* сломает экран смартфона 375px! */
  height: 200px;  /* длинный текст вывалится наружу! */
}

/* ✅ НАДЕЖНО: гибкий адаптивный блок */
.good-card {
  max-width: 500px; /* не шире 500px на ПК */
  width: 100%;      /* плавно сожмется на телефоне */
}
```

ЧЕГО СТРОГО ИЗБЕГАТЬ

- **Ловушка width: 500px**
  Экран телефона имеет ширину всего 375-390px. Жесткие 500px вылезут за край и сломают страницу.

- **Спасение max-width**
  Свойство max-width: 500px на мониторе ограничит ширину, а на телефоне карточка мягко сожмется под экран.

- **Не фиксируйте height**
  Если текста в описании окажется больше, слова вывалятся из рамки карточки и перекроют соседей.

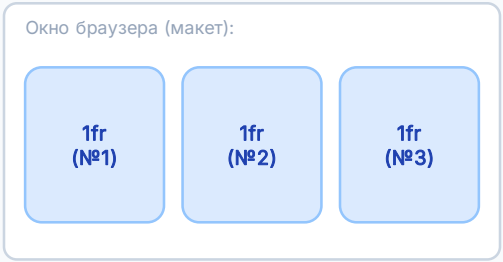
- **Пусть контент дышит**
  Задавайте карточкам внутренние отступы padding, а правильную высоту браузер всегда посчитает сам!

Почему на карточках товаров опасно жестко фиксировать ширину и высоту в пикселях?

СХЕМА: АДАПТИВНОСТЬ

1. КОМПЬЮТЕР (>1024px)

Широкий монитор: 3 колонки в ряд



КОМПЬЮТЕР (>1024px):

3 карточки рецептов в ряд. Страница свободно заполняет весь экран монитора.

- Поведение: элементы свободно помещаются в один широкий ряд.
- **Правило в CSS:**

```
grid-template-columns: repeat(3, 1fr);
```

2. ПЛАНШЕТ (768–1024px)

Планшет: автоматическая сетка 2×2



ПЛАНШЕТ (768–1024px):

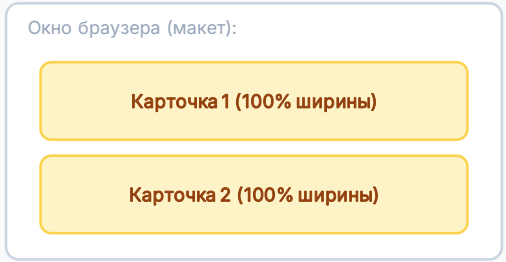
Карточки автоматически перестраиваются в 2 колонки и становятся крупнее.

- Поведение: сетка переносит карточки в 2 ряда по 50% ширины.
- **Правило в CSS:**

```
@media (max-width: 1024px) → repeat(2, 1fr)
```

3. СМАРТФОН (<768px)

Телефон: 1 колонка во всю ширину



СМАРТФОН (<768px):

1 колонка во всю ширину экрана. Текст крупный, кнопки удобны под палец.

- Поведение: карточки выстраиваются в вертикальную ленту на 100%.
- **Правило в CSS:**

```
@media (max-width: 768px) → 1fr
```

Как плавно перестроить 3 колонки каталога в 2 и 1 при сужении экрана?

ПРАКТИКА: ШАГ 1

index.html (карточка блюда)

```
<article class="recipe-card">
  <img src="img/pasta.jpg" alt="Паста Карбонара">
  <div class="card-body">
    <h3>Паста Карбонара</h3>
    <p>Классический рецепт со сливками</p>
    <button class="btn">Приготовить</button>
  </div>
</article>
```

ЧИСТАЯ СТРУКТУРА КАРТОЧКИ

- **Тег <article>**
  Семантически обозначает самостоятельную карточку блюда, которую можно переиспользовать в любом месте.
- **Фотография блюда**
  Тег <img> отображает аппетитное превью рецепта в самом верху карточки.
- **Контейнер .card-body**
  Внутренняя обертка для текста и кнопки, которой мы зададим аккуратный отступ padding.
- **Кнопка действия**
  Тег <button> с классом .btn для заказа или открытия рецепта — его мы стилизуем на следующих шагах.

Зачем для карточки блюда использовать тег <article>, а не обычный ?

ПРАКТИКА: ШАГ 2

css/style.css (шапка сайта)

```css
.site-header {
  background: #ffffff;
  border-bottom: 1px solid #e5e7eb;
}
.header-inner {
  display: flex;                  /* в один ряд */
  justify-content: space-between; /* растолкать по краям */
  align-items: center;             /* ровно по высоте */
  padding: 16px 24px;
}
```

ВЕРСТАЕМ ИДЕАЛЬНУЮ ШАПКУ

- **Оболочка шапки**
  Белый фон и тонкая нижняя рамка #e5e7eb четко отделяют шапку от остальной страницы.
- **Включаем ряд display: flex**
  Логотип слева и навигация справа мгновенно встают на одну горизонтальную линию.
- **Расталкиваем space-between**
  Логотип прижимается к левому краю, меню — к правому краю страницы.
- **Ровняем по высоте center**
  Логотип и текст ссылок идеально центрируются по вертикали без лишних margin.

Какие 3 строчки Flexbox создают классическую шапку с логотипом слева и меню справа?

ПРАКТИКА: ШАГ 3

css/style.css (сетка каталога)

```
.catalog-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px;                  /* равный зазор 24px */
  max-width: 1200px;          /* не шире 1200px */
  margin: 0 auto;             /* ровно по центру экрана */
  padding: 0 20px;            /* защита от краев */
}
```

ВИТРИНА ИЗ КАРТОЧЕК

- **Включаем Grid**
  Контейнер .catalog-grid становится удобной сеткой для всех расположенных внутри карточек.
- **repeat(3, 1fr)**
  Браузер делит ширину на 3 абсолютно равные колонки для красивой демонстрации рецептов.
- **Честный gap: 24px**
  Между карточками блюд появляется одинаковое расстояние, а по внешним краям ничего не торчит.
- **Центрирование 0 auto**
  Каталог держится строго посредине экрана даже на очень широких мониторах.

Как свойство margin: 0 auto удерживает сетку карточек по центру страницы?

ПРАКТИКА: ШАГ 4

css/style.css (карточка и hover)

```
.recipe-card {
  background: #ffffff;
  border-radius: 12px;        /* мягкие углы */
  overflow: hidden;             /* скруглить углы фото */
  border: 1px solid #e5e7eb;
  transition: transform 0.2s; /* плавный подъем */
}
.recipe-card:hover {
  transform: translateY(-4px);     /* всплытие вверх */
}
```

КРАСИВАЯ КАРТОЧКА И ОТКЛИК

- **Скругление углов 12px**
  Делает карточку современной и мягкой, убирая резкие прямоугольные углы.
- **Магия overflow: hidden**
  Обрезает прямые углы вложенной картинки строго по радиусу карточки, чтобы фото не торчало.
- **Эффект при наведении :hover**
  При наведении мышки карточка плавно всплывает вверх на 4px, приглашая кликнуть.
- **Плавность transition: 0.2s**
  Делает анимацию мягкой и приятной для глаз, исключая резкие скачки интерфейса.

Зачем карточке со скругленными углами нужно свойство overflow: hidden?

css/style.css (мобильная версия)

```css
@media (max-width: 768px) {
  /* Шапка переходит в вертикальный столбик */
  .header-inner {
    flex-direction: column;
    gap: 12px;
  }
  /* Каталог разворачивается в 1 колонку */
  .catalog-grid {
    grid-template-columns: 1fr;
  }
}
```

ИДЕАЛЬНЫЙ ВИД НА СМАРТФОНЕ

- **Порог 768px**
  Экраны планшетов и смартфонов как раз меньше 768px — это стандартный порог перестроения.
- **Шапка в колонку**
  Логотип встает сверху, навигация под ним через flex-direction: column. Ничего не слипается!
- **Каталог в 1fr**
  Карточки вытягиваются во всю ширину мобильного экрана: фото крупные, текст легко читается.
- **Удобно пальцам**
  Кнопки становятся широкими, на них комфортно нажимать одной рукой прямо на ходу.

Как с помощью @media перестроить сетку каталога из 3 колонок в 1 на экране телефона?

css/style.css (подключаем цвета)

```css
:root {
  --primary: #fe6002;       /* главный акцент */
  --primary-hover: #e05300; /* цвет при наведении */
}
.btn {
  background: var(--primary); color: #fff;
  border-radius: 8px;
}
.btn:hover {
  background: var(--primary-hover);
}
```

ФИНАЛЬНЫЙ ШТРИХ ДИЗАЙНА

- **Палитра в одном месте**
  Главный акцентный цвет объявлен в самом верху файла в блоке :root.
- **Кнопка берет var()**
  Кнопка заказа берет цвет из переменной var(--primary), не дублируя код вручную.
- **Реакция на наведение**
  При наведении курсора кнопка плавно темнеет на один тон благодаря var(--primary-hover).
- **Чистота проекта**
  В коде нет разброса случайных цветовых кодов, стили легко читать и поддерживать.

В чем преимущество выноса фирменного цвета в переменную :root перед ручной вставкой HEX?

ИТОГИ ЗАНЯТИЯ

Памятка разработчика

```
/* Золотые правила современной вёрстки: */

/* 1. Элементы в один ряд (шапка, меню, кнопки): */
display: flex; justify-content: space-between;

/* 2. Сетка из карточек (каталог, фотогалерея): */
display: grid; grid-template-columns: repeat(3, 1fr);

/* 3. Отступы между блоками: */
gap: 20px; /* без костылей margin! */

/* 4. Проверка на телефоне: */
@media (max-width: 768px) { ... }
```

ГЛАВНЫЕ ВЫВОДЫ ТЕМЫ

- **Flexbox для рядов**
  Идеален для выстраивания элементов в одну линию (шапка сайта, меню, аватар с текстом).
- **Grid для каталогов**
  Незаменим для сеток из карточек, делит ширину на равные аккуратные доли fr.
- **Отступы через gap**
  Делает одинаковые промежутки между элементами без костылей и лишних полей по краям.
- **Проверка на смартфоне**
  Всегда сужайте окно в браузере или используйте @media, чтобы сайт был удобен на телефоне.

Как кратко сформулировать главное отличие между Flexbox и CSS Grid?

РАЗБОР ОШИБОК

4 ЧАСТЫЕ ОШИБКИ НАЧИНАЮЩИХ ПРИ ВЕРСТКЕ

display: flex на ребенке вместо родителя

✕ **Ошибка:** Начинающий студент пишет `display: flex` на самой карточке и удивляется, почему карточки не встали в ряд.

✓ **Решение:** Объявлять `display: flex` СТРОГО на общем контейнере-родителе, где лежат карточки.

Двигать блоки случайными margin

✕ **Ошибка:** Попытка отодвинуть карточки друг от друга через хаотичные `margin-left` и `margin-right` наугад.

✓ **Решение:** Использовать современное свойство `gap: 20px` на флекс- или грид-контейнере.

Жесткая высота карточки height: 150px

✕ **Ошибка:** Жесткий `height` приводит к тому, что при изменении текста слова вываливаются за рамку карточки.

✓ **Решение:** Не фиксировать `height` вовсе (высота вырастет сама по контенту) или задать min-`height`.

Забыть проверить верстку на смартфоне

✕ **Ошибка:** Сайт верстается только под монитор ноутбука, а на смартфоне появляется неприятный боковой скролл.

✓ **Решение:** Всегда добавлять медиа-запрос @media (`max-width: 768px`) и тестировать сайт на телефоне.

Почему display: flex нужно объявлять на родительском теге, а не на самих карточках?

МЕЖДУНАРОДНЫЙ
ВОСТОЧНО-ЕВРОПЕЙСКИЙ
КОЛЛЕДЖ

ДО ВСТРЕЧИ!

УВИДИМСЯ НА СЛЕДУЮЩЕМ ЗАНЯТИИ